Bootloaders and firmware



© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!



Introduction



Bootloader role

- The bootloader is a piece of code responsible for
  - Basic hardware initialization
  - Loading of an application binary, usually an operating system kernel, from flash storage, from the network, or from another type of non-volatile storage.
  - Possibly decompression of the application binary
  - Execution of the application
- Besides these basic functions, most bootloaders provide a shell or menu
  - Menu to select the operating system to load
  - Shell with commands to load data from storage or network, inspect memory, perform hardware testing/diagnostics
- The first piece of code running by the processor that can be modified by us developers.



Booting on x86 platforms



Legacy BIOS booting (1)

- x86 platforms shipped before 2005-2006 include a firmware called *BIOS*
  - BIOS = Basic Input Output System
  - Part of the hardware platform, closed-source, rarely modifiable
  - Implements the booting process
  - Provides runtime services that can be invoked - not commonly used
  - Stored in some flash memory, outside of regular user-accessible storage devices
- To be bootable, the first sector of a storage device is “special”
  - MBR = Master Boot Record
  - Contains the partition table
  - Contains up to 446 bytes of bootloader code, loaded into RAM and executed
  - The BIOS is responsible for the RAM initialization
- https://en.wikipedia.org/wiki/BIOS

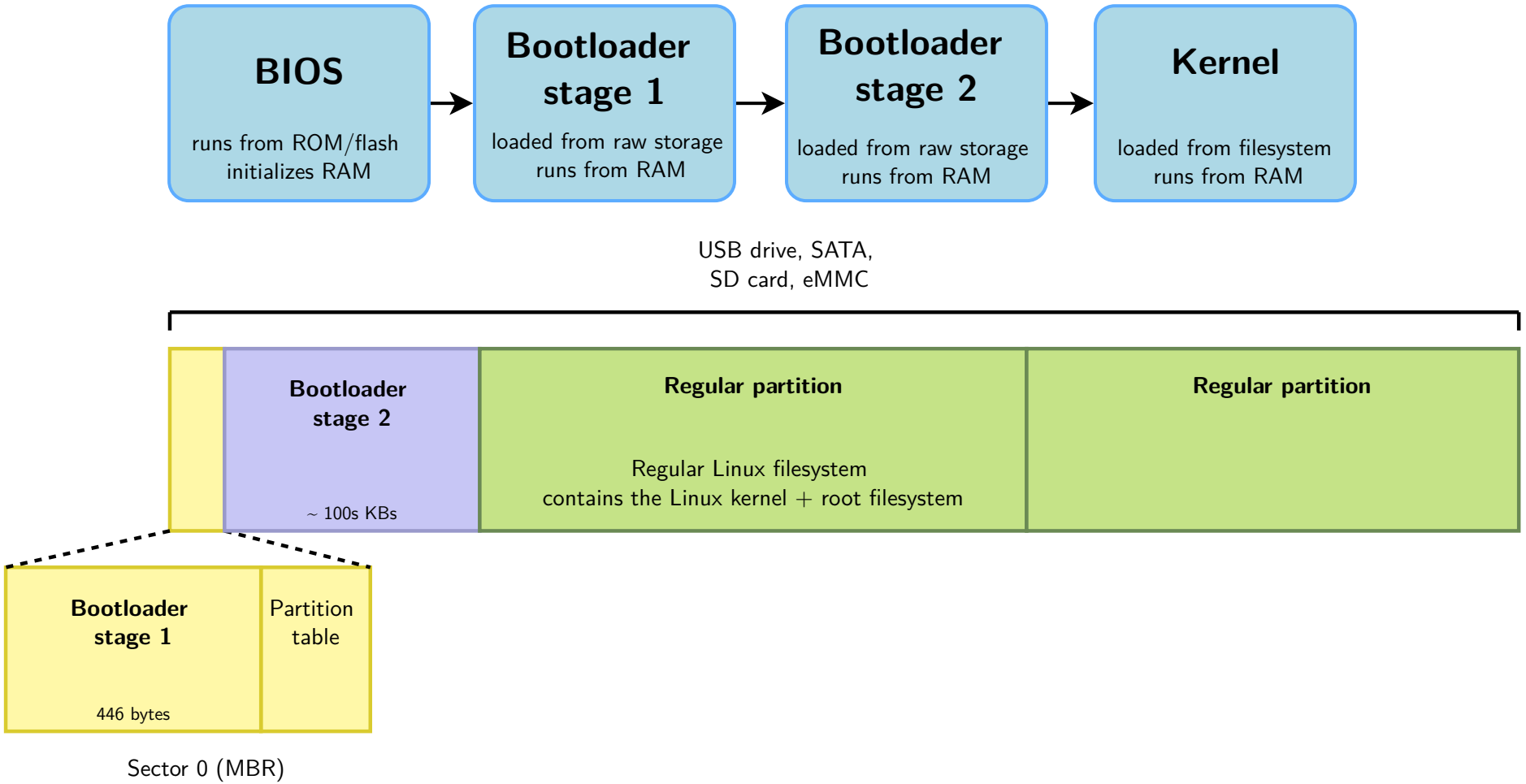


Legacy BIOS booting (2)

- Due to the limitation in size of the bootloader, bootloaders are split into two stages
  - Stage 1, which fits within the 446 bytes constraint
  - Stage 2, which is loaded by stage 1, and can therefore be bigger
- Stage 2 is typically stored outside of any filesystem, at a fixed offset → simpler to load by stage 1
- Stage 2 generally has filesystem support, so it can load the kernel image from a filesystem



Legacy BIOS booting: sequence and storage



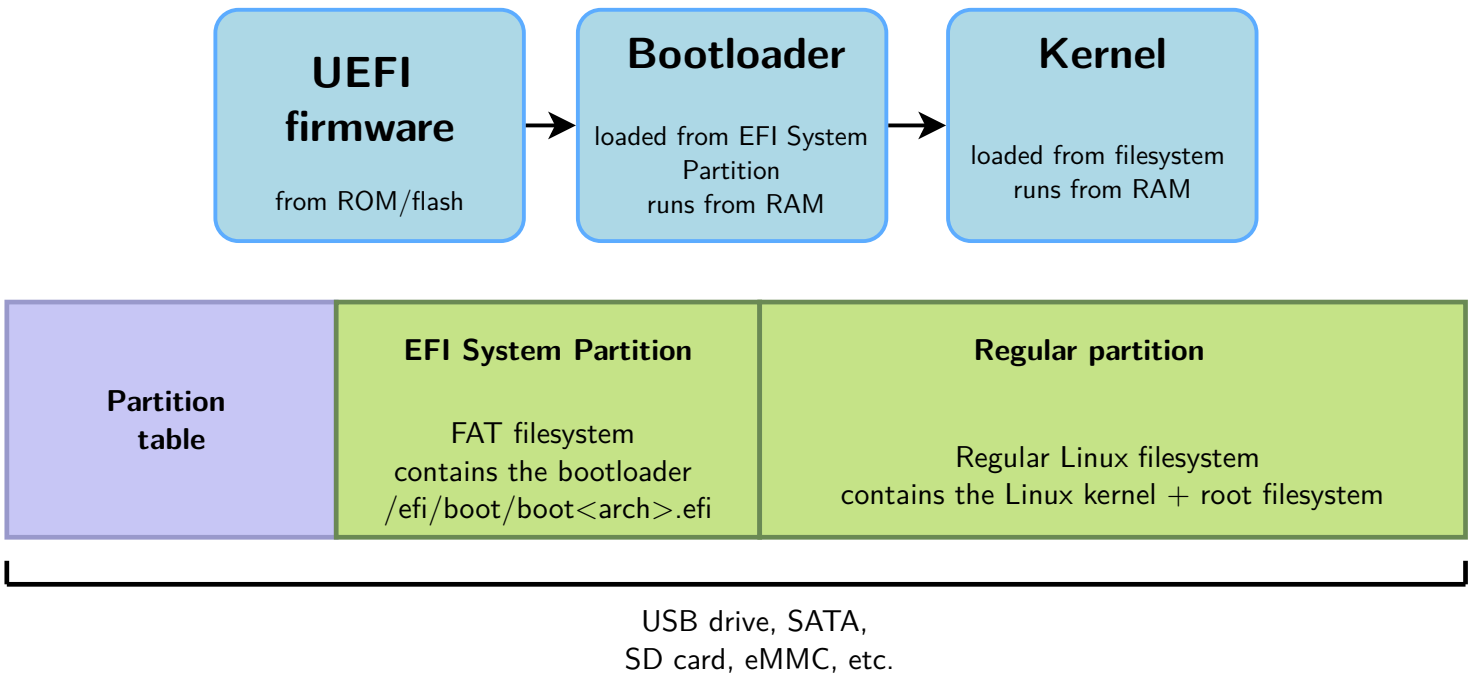


UEFI booting

- Starting from 2005-2006, UEFI is the new firmware interface on x86 platforms
  - Unified Extensible Firmware Interface
  - Describes the interface between the operating system and the firmware
  - Firmware in charge of booting
  - Firmware also provides runtime services to the operating system
  - Stored in some flash memory, outside of regular user-accessible storage devices
- Loads EFI binaries from the *EFI System Partition*
  - Generally a bootloader
  - Can also be directly the Linux kernel, with an *EFI Boot Stub*
- Special partition, formatted with the *FAT* filesystem
  - MBR: identified by type `0xEF`
  - GPT: identified with a specific *globally unique identifier*
- File `/efi/boot/bootx32.efi`, `/efi/boot/bootx64.efi`
- https://en.wikipedia.org/wiki/UEFI



UEFI booting: sequence and storage





ACPI

- Advanced Configuration and Power Interface
- *Open standard that operating systems can use to discover and configure computer hardware components, to perform power management, to perform auto configuration, and to perform status monitoring*
- *Tables* with descriptions of the hardware that cannot be dynamically discovered at runtime
- Tables provided by the firmware (UEFI or legacy) and used by the operating system (Linux kernel in our case)
- https://en.wikipedia.org/wiki/Advanced_Configuration_and_Power_Interface



UEFI and ACPI on ARM

- Historically UEFI and ACPI are technologies coming from the Intel/x86 world
- ARM is also pushing for the adoption of UEFI and ACPI as part of its *ARM System Ready* certification
  - Mainly for servers/workstations SoCs
  - Does not impact embedded SoCs
  - Currently not common in embedded Linux projects on ARM
  - https://www.arm.com/architecture/system-architectures/systemready-certification-program
- Also some on-going effort to use UEFI on RISC-V, but not the de-facto standard
- When an embedded platform uses UEFI → its booting process is similar to an *x86* platform



Booting on embedded platforms



Booting on embedded platforms: ROM code

- Most embedded processors include a **ROM code** that implements the initial step of the boot process
- The ROM code is written by the processor vendor and directly built into the processor
  - Cannot be changed or updated
  - Its behavior is described in the processor datasheet
- Responsible for finding a suitable bootloader, loading it and running it
  - From NAND/NOR flash, from USB, from SD card, from eMMC, etc.
  - Well defined location/format
- *Generally* runs with the external RAM not initialized, so it can only load the bootloader into an internal SRAM
  - Limited size of the bootloader, due to the size of the SRAM
  - Forces the boot process to be split in two steps: first stage bootloader (small, runs from SRAM, initializes external DRAM), second stage bootloader (larger, runs from external DRAM)



Booting on STM32MP1: datasheet

7 Boot configuration

7.1 Boot mode selection

In the STM32MP15x lines devices, different boot modes can be selected by means of the BOOT[2:0] pins. the reserved configuration is highlighted in gray in the table.

Table 13. Boot modes

BOOT2	BOOT1	BOOT0	Initial boot mode	Comments
0	0	0	UART and USB(1)	Wait incoming connection on: – USART2/3/6 and UART4/5/7/8 on default pins – USB High-Speed device on OTG_HS_DP/DM pins(2)
0	0	1	Serial NOR-Flash(3)	Serial NOR-Flash on QUADSPI(5)
0	1	0	eMMC™(3)	eMMC™ on SDMMC2 (default)(5)(6)
0	1	1	NAND-Flash(3)	SLC NAND-Flash on FMC
1	0	0	Engineering boot (No Flash boot)	Used to get debug access without boot from Flash(4)
1	0	1	SD-Card(3)	SD-Card on SDMMC1 (default)(5)(6)
1	1	0	UART and USB(1)(3)	Wait incoming connection on: – USART2/3/6 and UART4/5/7/8 on default pins – USB High-speed device on OTG_HS_DP/DM pins(2)
1	1	1	Serial NAND-Flash(3)	Serial NAND-Flash on QUADSPI(5)

Source:

Getting started with STM32MP157

Useful details:

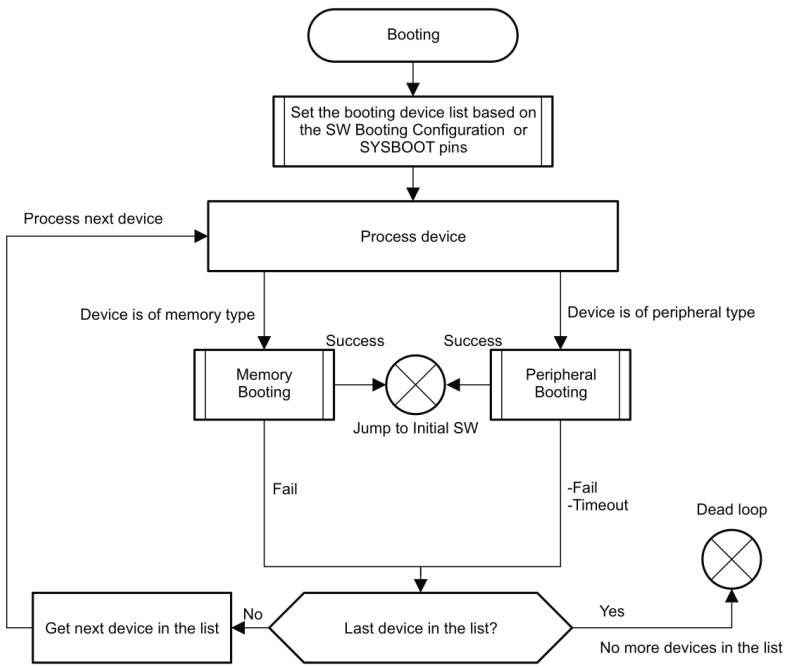
STM32 MPU ROM code overview

26.1.5 Booting

26.1.5.1 Overview

Figure 26-6 shows the booting procedure. First a **booting device list** is created. The list consists of all devices which will be searched for a booting image. The list is filled in based on the **SYSBOOT**.

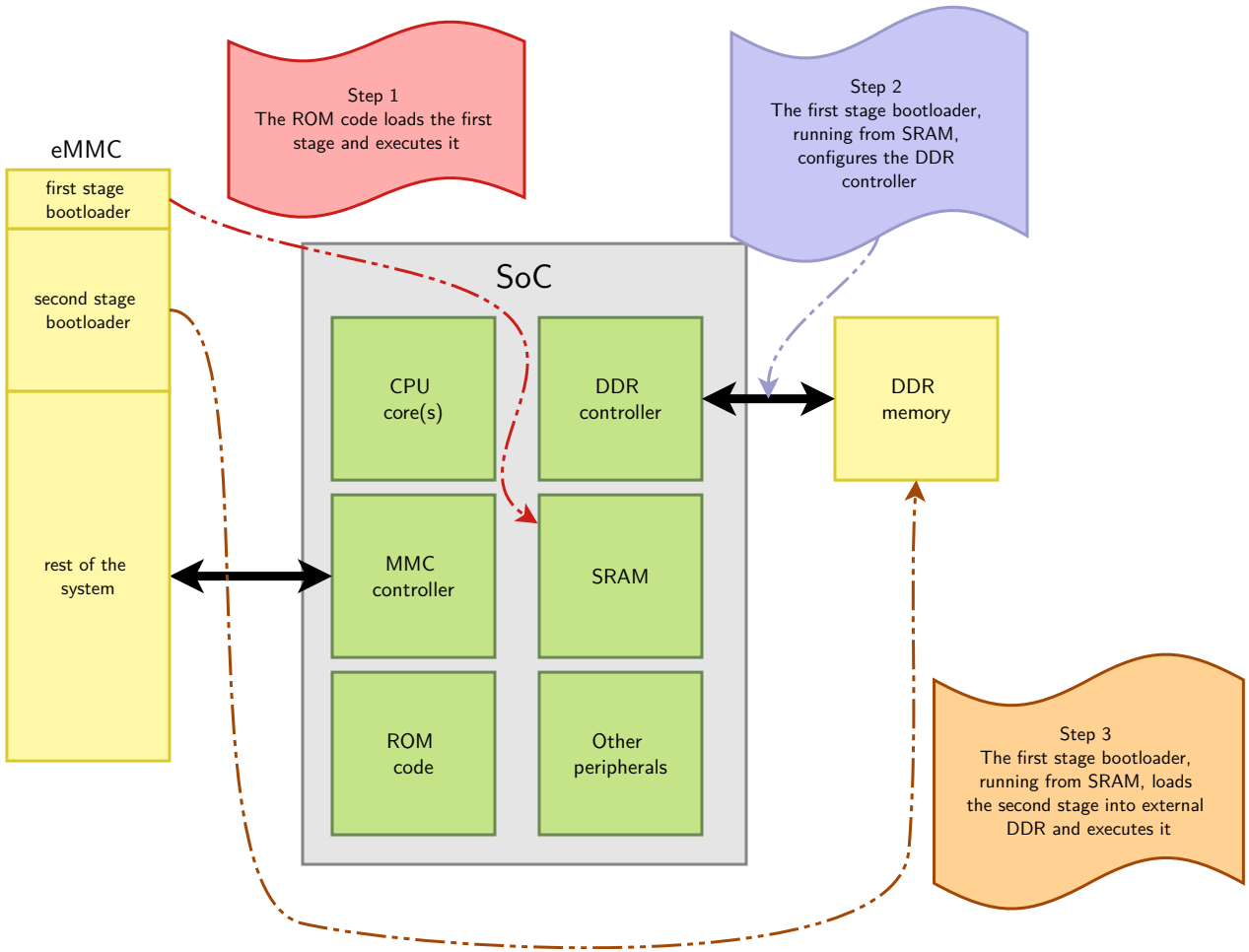
Figure 26-6. ROM Code Booting Procedure



Once the booting device list is set up, the booting routine examines the devices enumerated in the list sequentially and either executes the memory booting or peripheral booting procedure depending on the booting device type. The memory booting procedure is executed when the booting device type is one of NOR, NAND, MMC or SPI-EEPROM. The peripheral booting is executed when the booting device type is Ethernet, USB or UART.

Source:
AM335x TRM / chapter 26

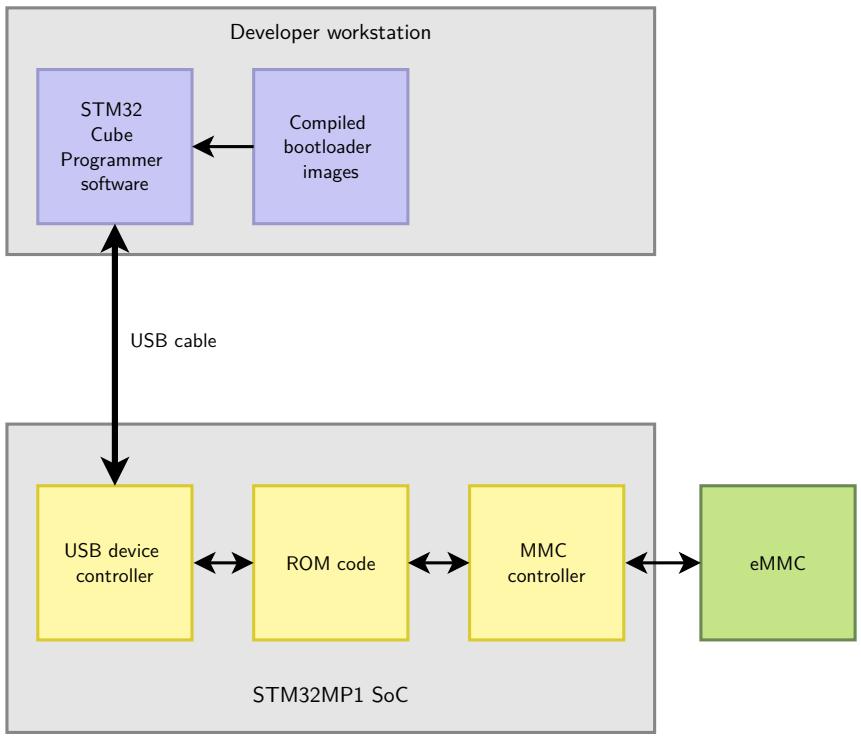
Two stage booting sequence





ROM code recovery mechanism

- Most ROM code also provide some sort of *recovery* mechanism, allowing to flash a board with no bootloader or a broken one, usually with a vendor-specific protocol over UART or USB.
- Often allows to push a new bootloader into RAM, making it possible to reflash the bootloader.
- Vendor-specific tools to run on the workstation
  - STM32MP1: STM32 Cube Programmer
  - NXP i.MX: uuu
  - Microchip AT91/SAM: SAM-BA
  - Allwinner: sunxi-fel
  - Some open-source, some proprietary
- Snagboot: new vendor agnostic tool replacing the above ones: https://github.com/bootlin/snagboot



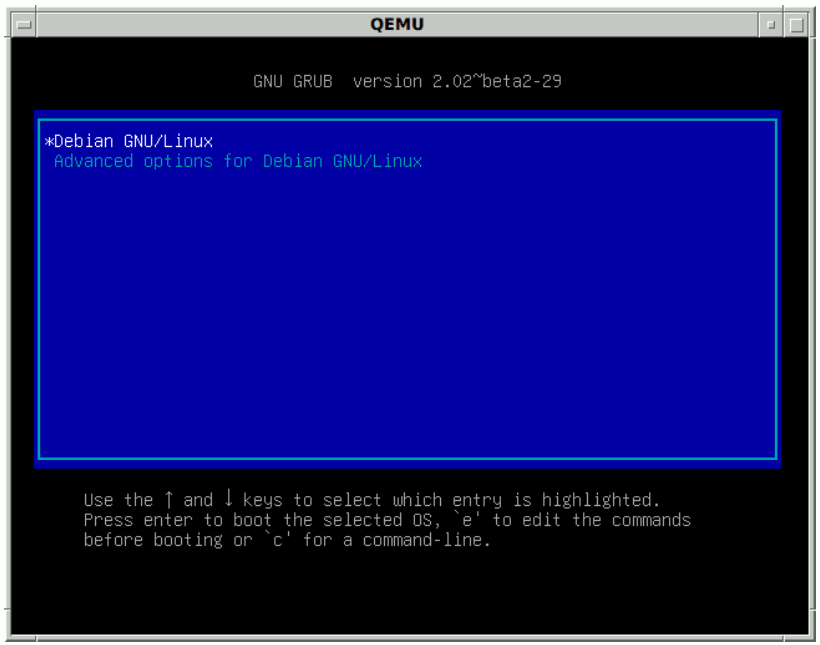


Bootloaders



GRUB

- *Grand Unified Bootloader*, from the GNU project
- De-facto standard in most Linux distributions for x86 platforms
- Supports x86 legacy and UEFI systems
- Can read many filesystem formats to load the kernel image, modules and configuration
- Provides a menu and powerful shell with various commands
- Can load kernel images over the network
- Also supports ARM, ARM64, RISC-V, PowerPC, but less popular than other bootloaders on those platforms
- https://www.gnu.org/software/grub/
- https://en.wikipedia.org/wiki/GNU_GRUB





Syslinux

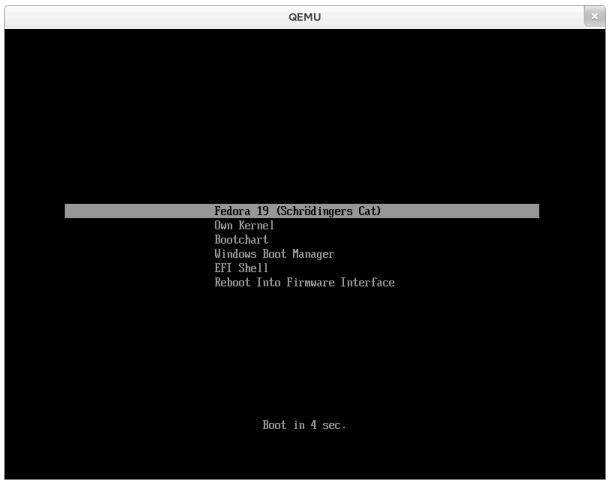
- For network and removable media booting (USB key, SD card, CD-ROM)
- `syslinux`: booting from FAT filesystem
- `pxelinux`: booting from the network
- `isolinux`: booting from CD-ROM
- `extlinux`: booting from numerous filesystem types
- A bit rustic to build and configure, not very actively maintained, but still useful for specific use-cases
- https://wiki.syslinux.org/
- https://kernel.org/pub/linux/utils/boot/syslinux/





systemd-boot

- Simple UEFI boot manager
- Useful alternative to GRUB for UEFI systems: simpler than GRUB
- Configured using files stored in the *EFI System Partition*
- Part of the *systemd* project, even though obviously distinct from *systemd* itself
  - See our slides later in this course for more details on *systemd*
- https://www.freedesktop.org/wiki/Software/systemd/systemd-boot/





shim

- Minimal UEFI bootloader
- Mainly used in secure boot scenario: it is signed by Microsoft and therefore successfully verified by UEFI firmware in the field
- Allows to chainload another bootloader (GRUB) or directly the Linux kernel, with signature checking
- https://github.com/rhboot/shim



U-Boot

- The de-facto standard and most widely used bootloader on embedded architectures: ARM, ARM64, RISC-V, PowerPC, MIPS, and more.
- Also supports x86 with UEFI firmware.
- Very likely the one provided by your SoC vendor, SoM vendor or board vendor for your hardware.
- We will study it in detail in the next section, and use it in all practical labs of this course.
- https://u-boot-project.org/





Barebox

- Another bootloader for most embedded CPU architectures: ARM/ARM64, MIPS, PowerPC, RISC-V, x86, etc.
- Initially developed as an alternative to U-Boot to address some U-Boot shortcomings
  - *kconfig* for the configuration like the Linux kernel
  - well-defined *device model* internally
  - More Linux-style shell interface
  - Cleaner code base
- Actively maintained and developed, but
  - Less widely used than U-Boot
  - Less platform support than in U-Boot
- https://www.barebox.org/
- Talk *barebox Bells and Whistles*, by Ahmad Fatoum, ELCE 2020, video and slides





Trusted firmware



Concept

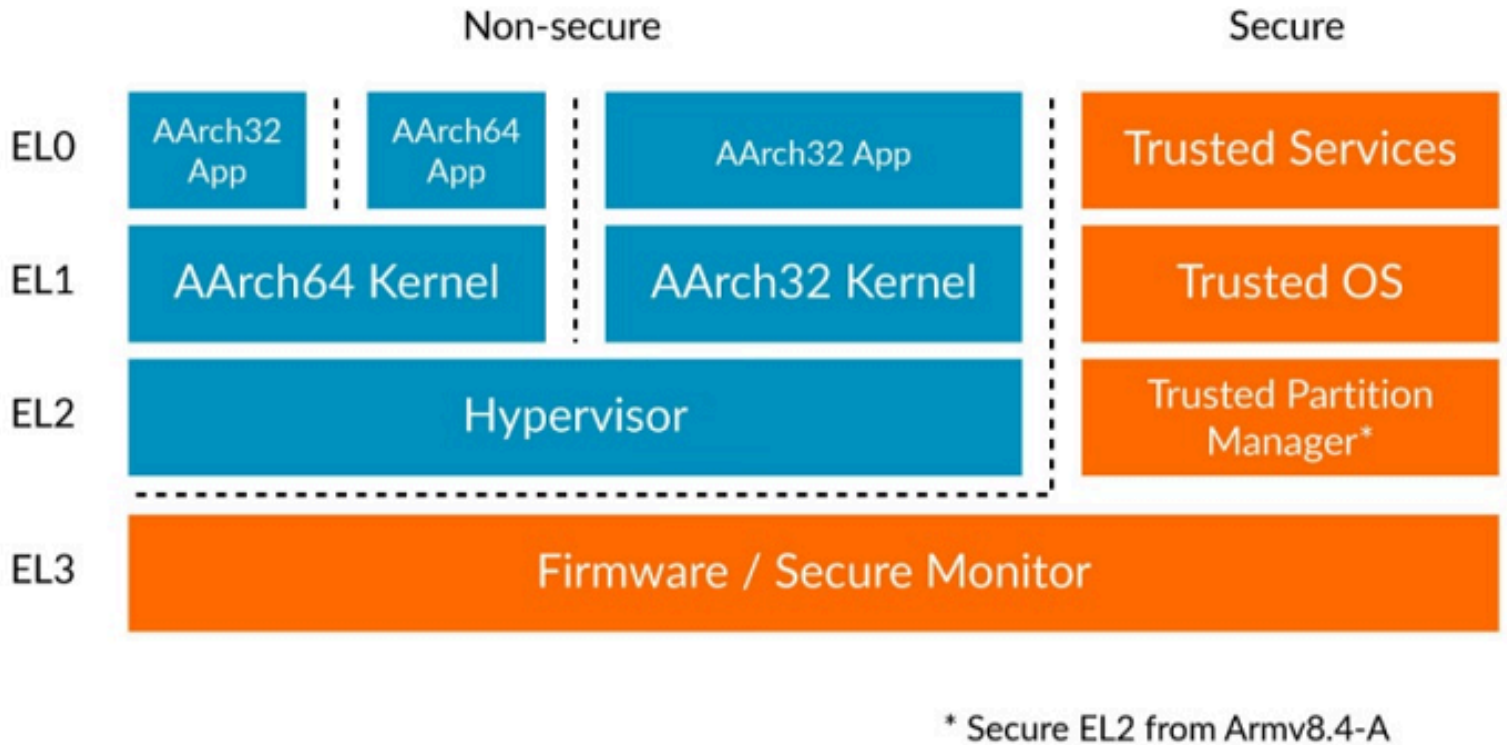
- Traditionally, bootloaders are only used during the booting process
  - Bootloader loads operating system, jumps to it, and is discarded
- Modern SoCs have advanced security mechanisms that require running some sort of *trusted firmware*
- This firmware is loaded by the bootloader, or part of the boot chain itself
- This *trusted firmware* **stays resident** after control has been passed to the OS
  - It is stored in a dedicated portion of the DDR, or some specific SRAM, inaccessible from the OS
  - It provides services to the OS, which the OS cannot perform directly
  - Can also be responsible for running a secure OS alongside the regular OS (Linux in our case)



ARM

- Modern ARMv7 and ARMv8 processors have
  - 4 privilege levels (*Exception Levels*)
    - EL3, the most privileged, runs secure firmware
    - EL2, typically used by hypervisors, for virtualization
    - EL1, used to run the Linux kernel
    - EL0, used to run Linux user-space applications
  - 2 *worlds*
    - Normal world, used to run a general purpose OS, like Linux
    - Secure world, to run a separate, isolated, secure operating system and applications. Also called *TrustZone* by ARM.
- EL3 only exists in the secure world
- EL2 exists in both secure and normal worlds since ARMv8.4, before that EL2 was only in the normal world
- EL1 and EL0 exist in both secure and normal worlds

ARM exception levels and worlds

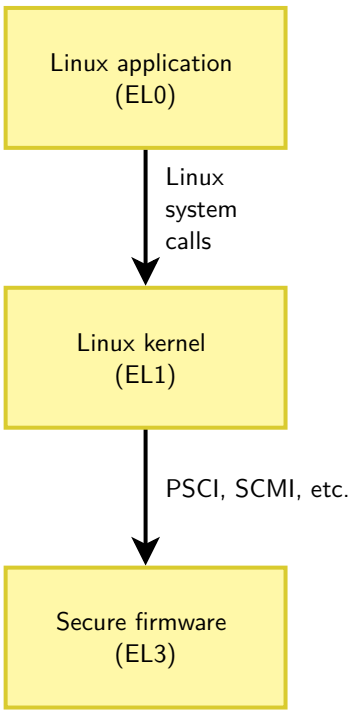


Source: ARM documentation



Interfaces with secure firmware

- Standardized by ARM
- Services
  - implemented by the secure firmware
  - called by the operating system
- Prevents the operating system running in normal world from directly accessing critical hardware resources
- PSCI, Power State Coordination Interface
  - Power management related: turn CPUs on/off, CPU idle state, platform shutdown/reset
- SCMI, System Control and Management Interface
  - Power domain, clocks, sensor, performance
- Secure firmware implementing these interfaces is
  - Mandatory to run Linux on ARMv8
  - Mandatory to run Linux on some ARMv7 platforms, but not all





TF-A

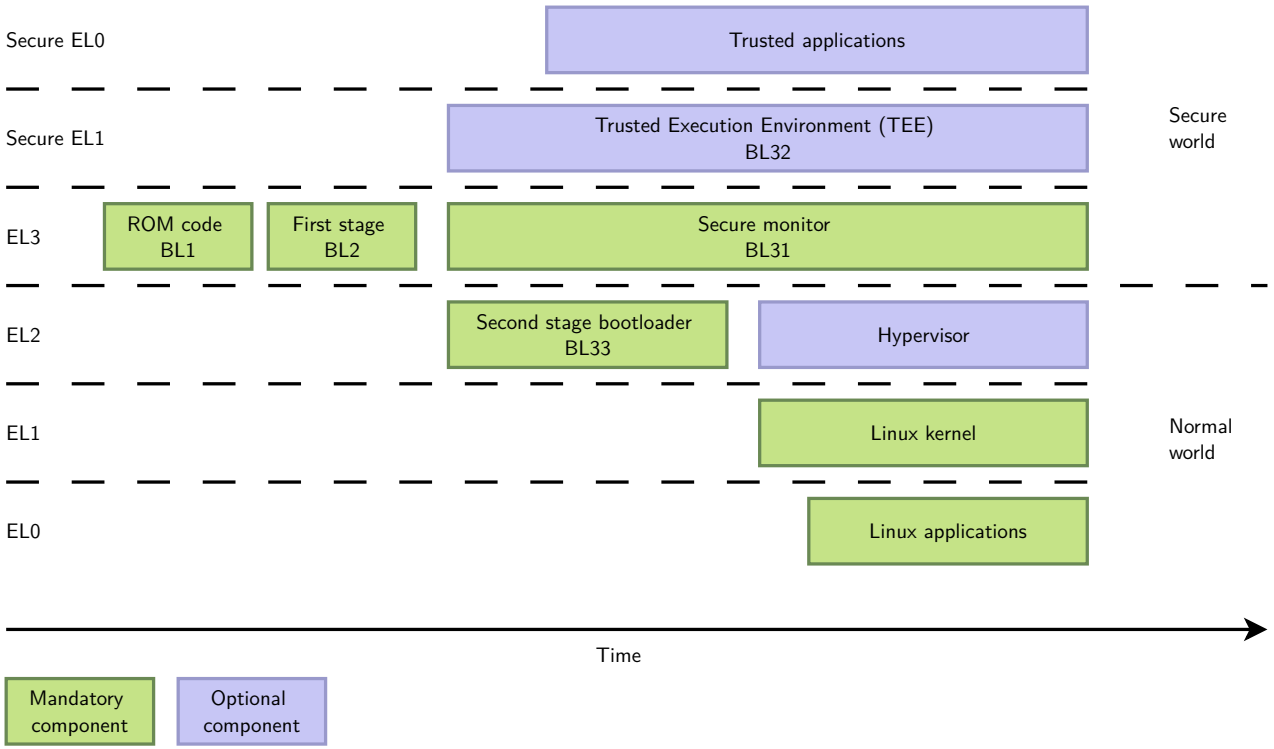
- *Trusted Firmware-A (TF-A) provides a reference implementation of secure world software for Armv7-A and Armv8-A, including a Secure Monitor executing at Exception Level 3 (EL3)*
- Formerly known as *ATF*, for ARM Trusted Firmware
- Implements the various standard interfaces that operating systems need from the secure firmware
- Has drivers for the hardware blocks that are not accessed directly by Linux
- Needs to be ported for each SoC
- Depending on the platform, may also need to be ported per board: DDR initialization
- Used on the vast majority of ARMv8 platforms, and on a few recent ARMv7 platforms
- https://www.trustedfirmware.org/projects/tf-a/



Trusted OS, OP-TEE

- A trusted operating system can run in the *secure world*, also called *Trusted Execution Environment* or *TEE*
- Hardware partitioning between *secure world* and *normal world*
  - Some hardware resources only available in the *secure world*, by the trusted OS
- Allows to run trusted applications/services
  - isolated from Linux
  - can provide services to Linux applications
- Most common open-source implementation: *OP-TEE*
  - Supported by most silicon vendors
  - https://www.op-tee.org/

ARM: summary

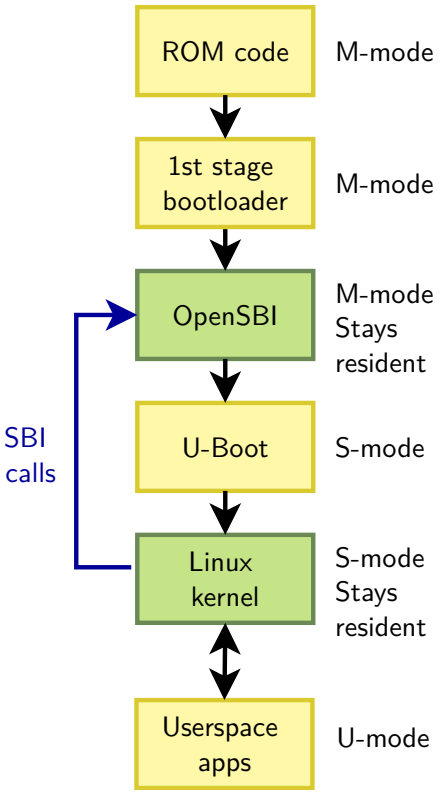


Largely inspired from *Ahmad Fatoum* presentation *From Reset Vector to Kernel*, slides, video
See also details about the ARM terms: BL1, BL2...



RISC-V

- Linux-class RISC-V processors have several privilege levels
  - M-mode: machine mode
  - S-mode: level at which the Linux kernel runs
  - U-mode: level at which Linux user-space applications run
- Some specific HW resources are not accessible in S-mode
- A more privileged firmware runs in M-mode
- RISC-V has defined SBI, *Supervisor Binary Interface*
  - Standardized interface between the OS and the firmware
  - https://github.com/riscv-non-isa/riscv-sbi-doc
- OpenSBI is a reference, open-source implementation of SBI
  - https://github.com/riscv-software-src/opensbi

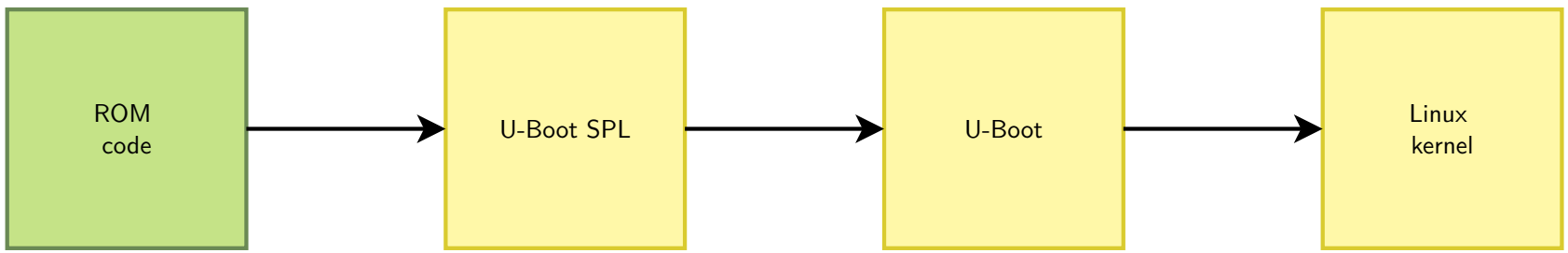




Example boot sequences on ARM



TI AM335x (32 bit BeagleBone): ARMv7



Stored in ROM

Loaded from a file called MLO in a FAT filesystem in the first bootable partition

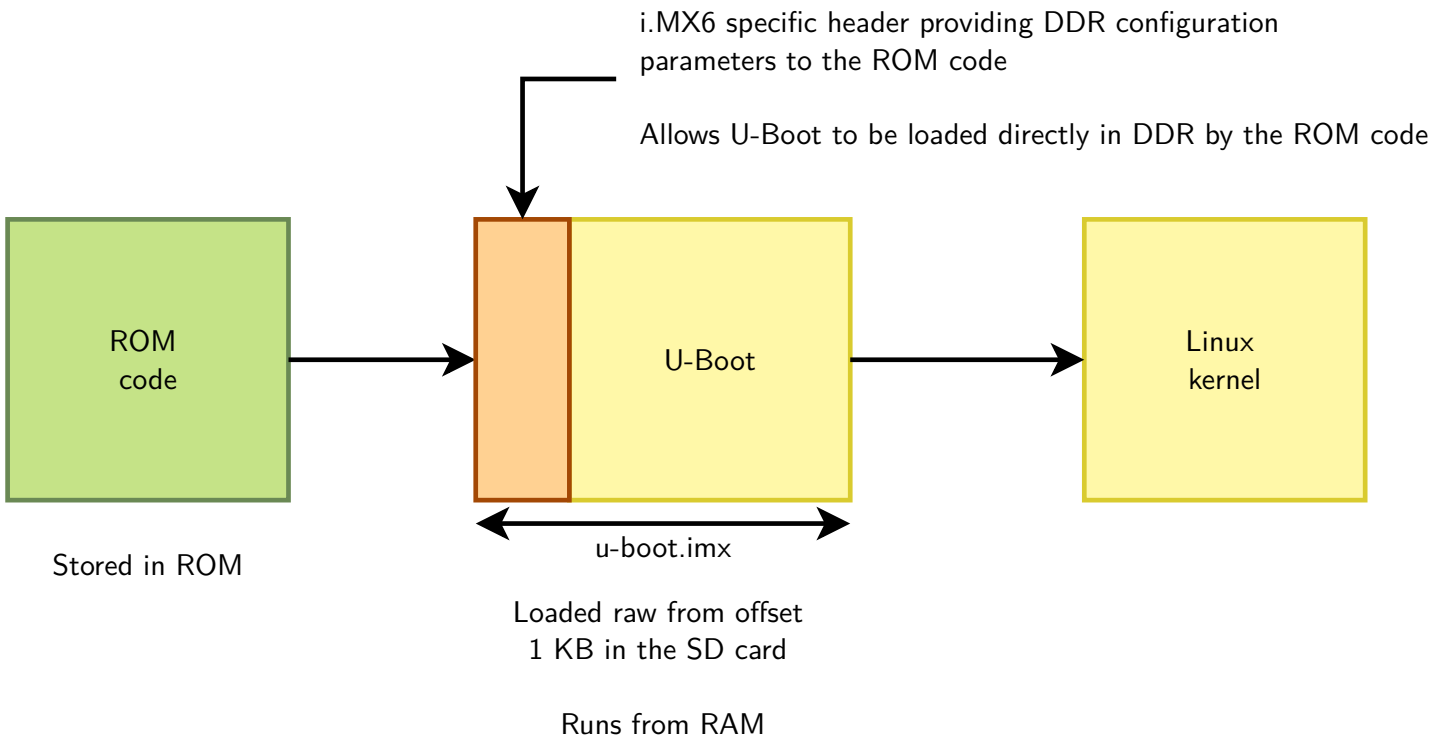
Runs from SRAM

Loaded from a file called u-boot.img in a FAT filesystem

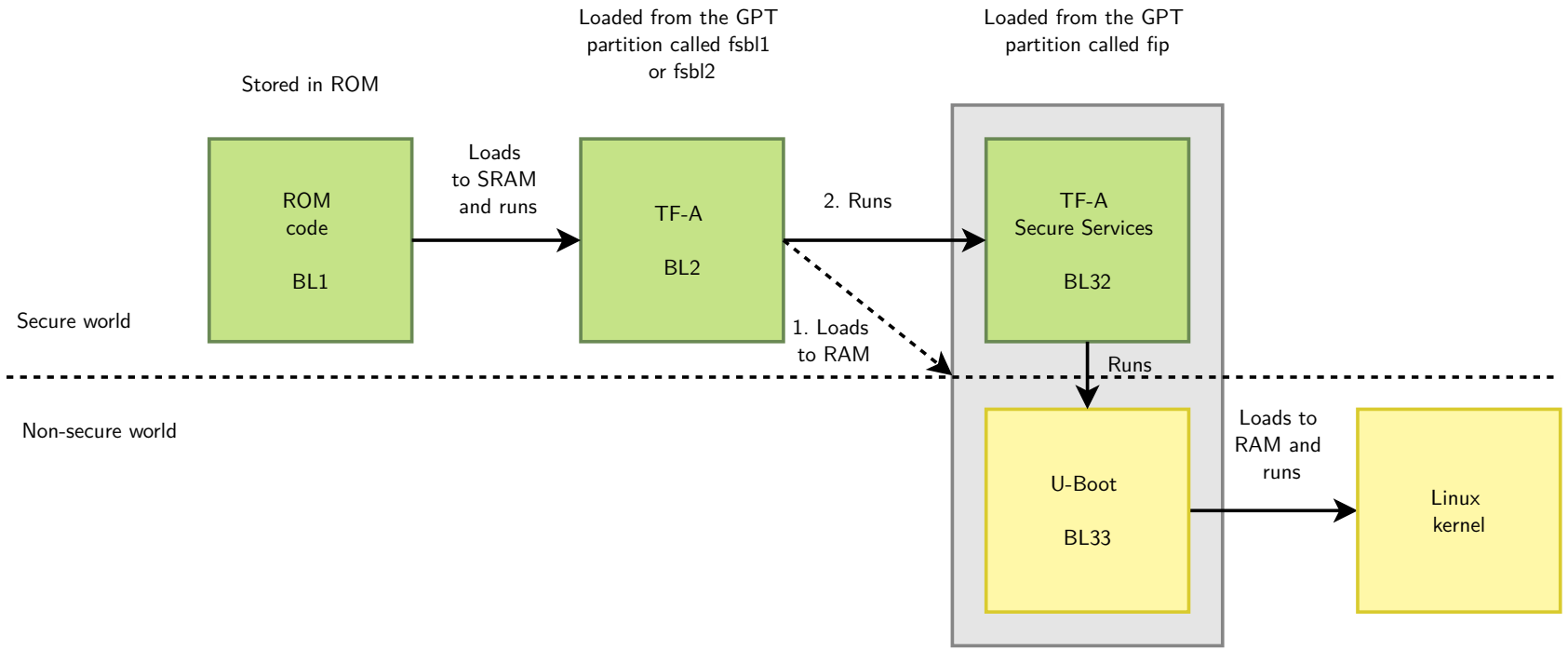
Runs from RAM



NXP i.MX6: ARMv7

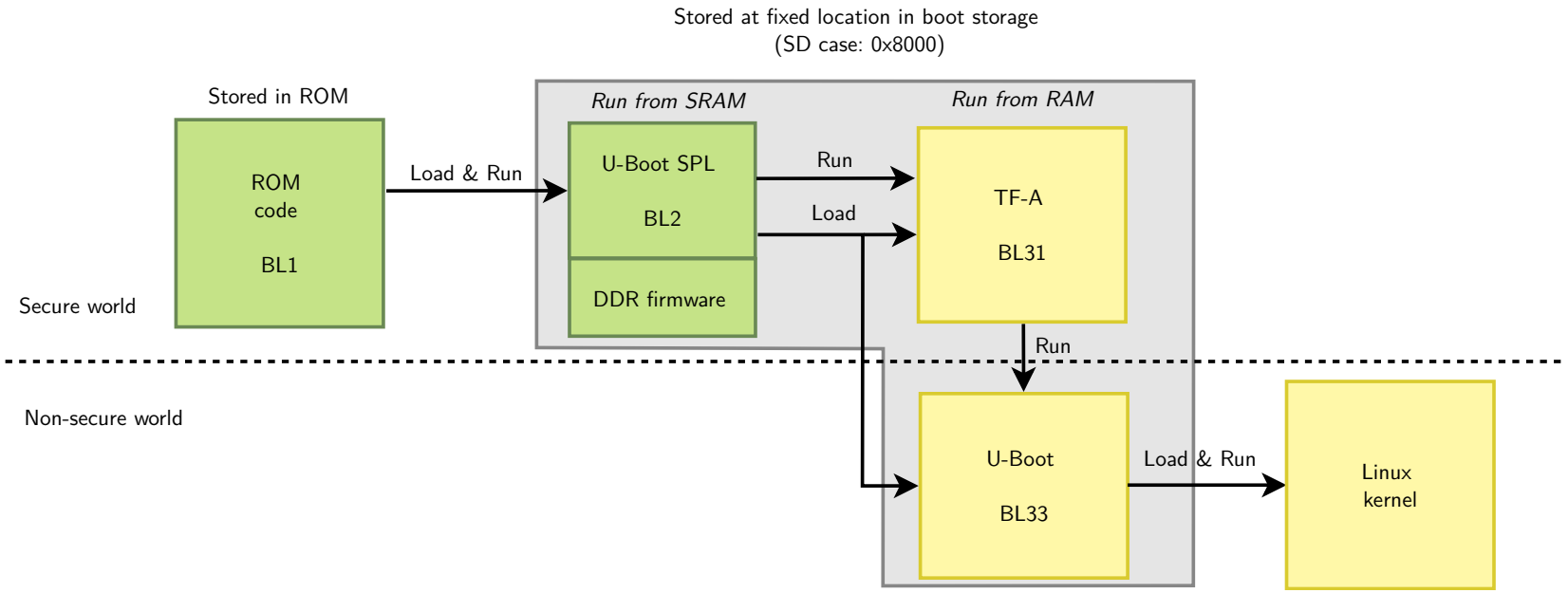


Note: this diagram shows one possible boot flow on NXP i.MX6, but it is also possible to use the U-Boot SPL → U-Boot boot flow on i.MX6.

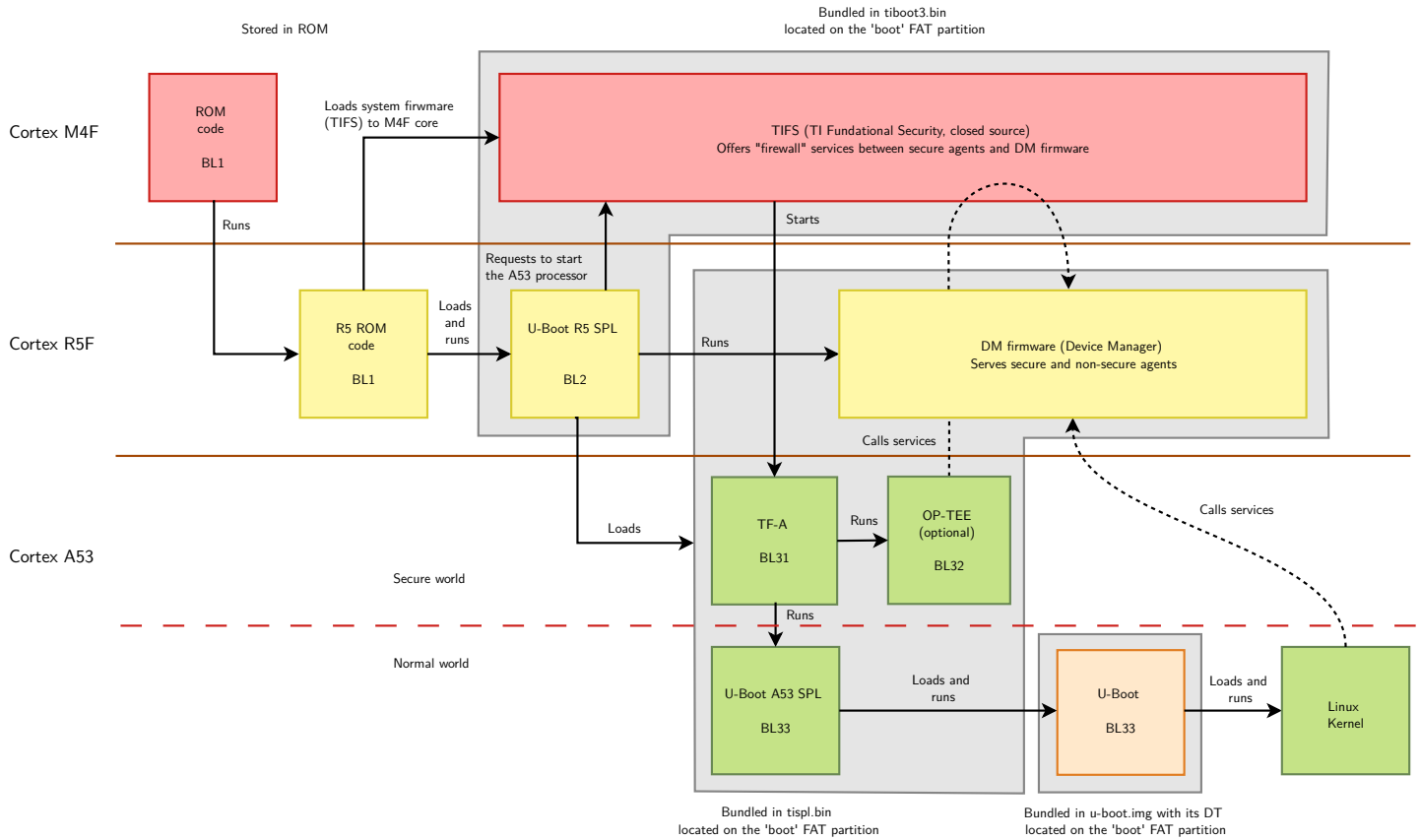


Note: booting with U-Boot SPL and U-Boot is also possible.

IMX93: ARMv8


Stored at fixed location in boot storage
(SD case: 0x8000)
Stored in ROM
Run from SRAM
Run from RAM
ROM
code
BL1
Load & Run
U-Boot SPL
BL2
DDR firmware
Run
Load
TF-A
BL31
Secure world
Run
Non-secure world
U-Boot
BL33
Load & Run
Linux
kernel

TI AM62x (BeaglePlay): ARMv7 and ARMv8 cores



See https://u-boot.readthedocs.io/en/latest/board/ti/am62x_sk.html for details.



The U-boot bootloader



U-Boot

U-Boot is a typical free software project

- License: GPLv2 (same as Linux)
- Freely available at: https://u-boot-project.org/
- Documentation available at: https://docs.u-boot-project.org/
- The latest development source code is available in a Git repository: https://git.u-boot-project.org/u-boot/u-boot.git
- Development and discussions happen around an open mailing-list: https://lists.u-boot-project.org/pipermail/u-boot/
- Follows a regular release schedule. Every 2 or 3 months, a new version is released. Versions are named `YYYY.MM`.

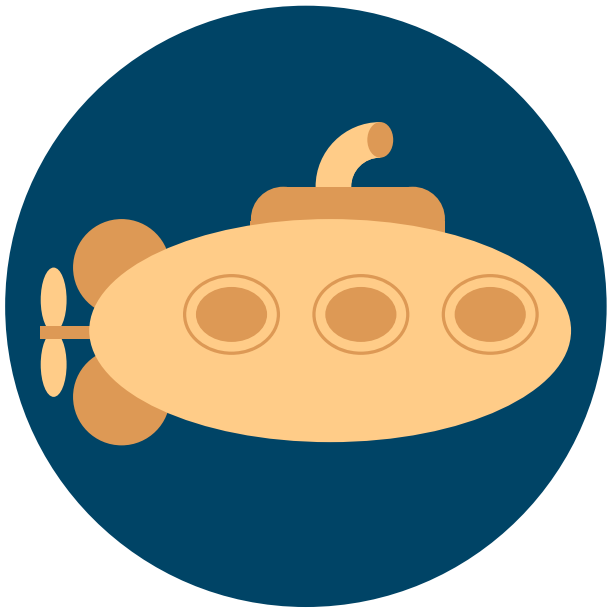


Image source



Where to get U-Boot from?

- **Ideal:** your platform is supported directly by **upstream** U-Boot
  - Best quality → code reviewed and approved by the community
  - Long-term maintenance
  - Use directly U-Boot from https://git.u-boot-project.org/u-boot/u-boot.git Git repository
- **Less ideal:** use a **fork** of U-Boot by your silicon vendor, system-on-module vendor or board vendor
  - Generally older, does not follow all upstream U-Boot updates
  - Changes not reviewed by the community → quality is often dubious
  - Check your HW vendor documentation/SDK
- If designing your own custom board
  - You will have to port U-Boot
  - If good support for your SoC in upstream U-Boot → use upstream U-Boot
  - If not → use the U-Boot fork from your SoC vendor



U-Boot configuration

- Configuration system based on *kconfig* from the Linux kernel
- The configs/ directory contains configuration files for supported boards or platforms
  - There may be a single configuration supporting multiple boards based on the same processor
  - The configuration files defines all relevant options: CPU type, drivers needed, U-Boot features to compile in
  - Examples:
    - configs/stm32mp15_basic_defconfig
    - configs/stm32mp15_trusted_defconfig
- Note: migration to *kconfig* is still on-going
  - Not all boards have been converted to the new configuration system.
  - Many boards still have a combination of configuration settings in include/configs/ header files, and configuration settings in `defconfig` files



U-Boot configuration file: stm32mp15_trusted_defconfig

```
CONFIG_ARM=y
CONFIG_ARCH_STM32MP=y
CONFIG_TFABOOT=y
CONFIG_SYS_MALLOC_F_LEN=0x3000
CONFIG_ENV_OFFSET=0x280000
CONFIG_ENV_SECT_SIZE=0x40000
CONFIG_DEFAULT_DEVICE_TREE="stm32mp157c-ev1"
[..]
CONFIG_CMD_ADTIMG=y
CONFIG_CMD_ERASEENV=y
CONFIG_CMD_NVEDIT_EFI=y
CONFIG_CMD_MEMINFO=y
CONFIG_CMD_MEMTEST=y
CONFIG_CMD_UNZIP=y
CONFIG_CMD_ADC=y
CONFIG_CMD_CLK=y
CONFIG_CMD_DFU=y
CONFIG_CMD_FUSE=y
CONFIG_CMD_GPIO=y
[...]
CONFIG_SPI=y
CONFIG_DM_SPI=y
CONFIG_STM32_QSPI=y
CONFIG_STM32_SPI=y
[...]
```

See the full file: configs/stm32mp15_trusted_defconfig

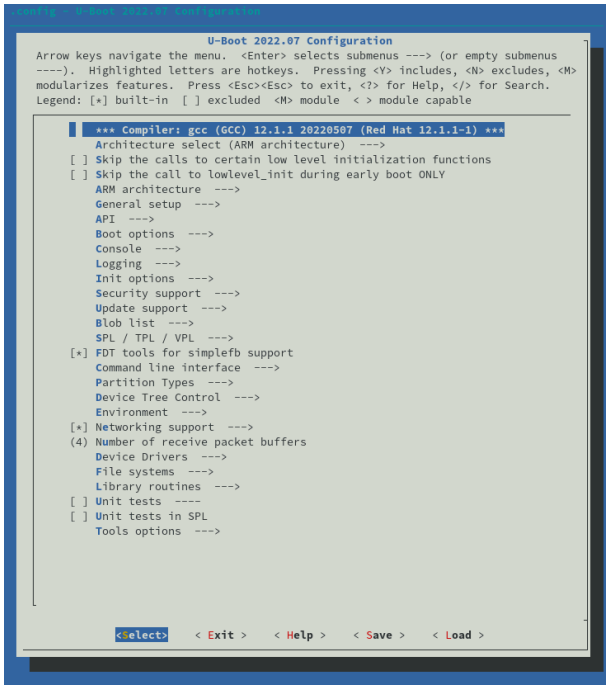


U-Boot configuration

- U-Boot must be configured before being compiled
- Configuration stored in a `.config` file
- Load a pre-defined configuration

```
$ make BOARDNAME_defconfig
```

  Where `BOARDNAME` is the name of a configuration, as visible in the configs/ directory.
- You can then run `make menuconfig` to further customize U-Boot's configuration.





U-Boot compilation

- The path to the cross-compiler must be specified in the `CROSS_COMPILE` variable
- `CROSS_COMPILE` contains the prefix common to all cross-compilation tools, e.g `arm-linux-`
- Common to add the cross-compiler location in `PATH` to keep the `CROSS_COMPILE` value short

```
$ export PATH=/path/to/toolchain/bin:$PATH
$ make CROSS_COMPILE=arm-linux-
```

- The main result is a `u-boot.bin` file, which is the U-Boot image.
- Depending on your specific platform, or what storage device you're booting from (NAND or MMC), there may be other specialized images: `u-boot.img`, `u-boot.kwb`...



Concept of U-Boot SPL

- To meet the need of a two-stage boot process, U-Boot has the concept of *U-Boot SPL*
- SPL = *Secondary Program Loader*
- The SPL is a stripped-down version of U-Boot, made small enough to meet the size constraints of a first stage bootloader
- Configured through *menuconfig*, one can define the subset of drivers to include
- No U-Boot shell/commands: the behavior is hardcoded in C code
- For some platforms: TPL, *Tertiary Program Loader*, an even more minimal first stage bootloader to do TPL → SPL → main U-Boot.



Device Tree in U-Boot

- The *Device Tree* is a data structure that describes the topology of the hardware
- Allows software to know which hardware peripherals are available and how they are connected to the system
- Initially mainly used by Linux, now also used by U-Boot, Barebox, TF-A, etc.
- Used by U-Boot on most platforms.
- Since v2024.07 the Device Tree files location depends on `CONFIG_OF_UPSTREAM`:
  - `dts/upstream/src/ARCH/VENDOR` when `CONFIG_OF_UPSTREAM` is set
  - `arch/ARCH/dts` otherwise
- One `.dts` for each board: need to create one if you build a custom board
- U-Boot *defconfigs* usually specify a default Device Tree, but it can be overridden using the `DEVICE_TREE` variable
- More details on the *Device Tree* later in this course.



U-Boot build example: TI AM335x BeagleBoneBlack wireless

- One *defconfig* file suitable for all AM335x platforms: configs/am335x_evm_defconfig
  - Yes its name looks like it supports only the EVM (EValuation Module) board
  - Contains `CONFIG_DEFAULT_DEVICE_TREE="am335x-evm"` → uses arch/arm/dts/am335x-evm.dts by default
- One *Device Tree* file describing the BeagleBoneBlack Wireless: arch/arm/dts/am335x-boneblack-wireless.dts
- Configure and build U-Boot

```
$ export PATH=/path/to/toolchain/bin:$PATH
$ make am335x_evm_defconfig
$ make DEVICE_TREE=am335x-boneblack-wireless CROSS_COMPILE=arm-linux-
```

- Produces:
  - `MLO`, the SPL, first-stage bootloader. Called MLO (*Mmc LOad*) as required on TI platforms.
  - `u-boot.img`, full U-Boot, second-stage bootloader



Installing U-Boot

1. If U-Boot is loaded from external storage, just update the binaries on such storage.
2. If U-Boot is loaded from internal storage (eMMC or NAND), you can update it using *Snagboot* (https://github.com/bootlin/snagboot) if it supports your SoC, or with the custom solution from the SoC vendor.
3. An alternative is to reflash internal storage with JTAG (if available), but that's more complicated and requires a JTAG probe.



U-boot shell prompt

- Connect the target to the host through a serial console.
- Power-up the board. On the serial console, you should see U-Boot starting up.
- The U-Boot shell offers a set of commands.
- The U-Boot shell is not a Linux shell: commands are completely different from Linux ones.

```
U-Boot SPL 2022.01 (Mar 31 2022 - 14:58:17 +0200)
Trying to boot from MMC1

U-Boot 2022.01 (Mar 31 2022 - 14:58:17 +0200)

CPU  : AM335X-GP rev 2.1
Model: TI AM335x BeagleBone Black DRAM:  512 MiB
WDT:   Started wdt@44e35000 with servicing (60s timeout)
NAND:  0 MiB
MMC:   OMAP SD/MMC: 0, OMAP SD/MMC: 1
Loading Environment from FAT... OK
Net:   Could not get PHY for ethernet@4a100000: addr 0
eth2: ethernet@4a100000, eth3: usb_ether [PRIME]
Hit any key to stop autoboot:  0
=>
```



U-Boot `help` command

- `help` command to list all available commands
- The set of available commands depend on the U-Boot configuration
  - Many `CONFIG_CMD_*` options to enable commands at compile time
  - See *Command line interface* submenu in `menuconfig`
- `help <command>` for the complete help of one command

```
STM32MP> help
?          - alias for 'help'
adc        - ADC sub-system
adtimg     - manipulate dtb/dtbo Android image
base       - print or set address offset
[...]
usb        - USB sub-system
[...]

STM32MP> help usb
usb - USB sub-system

Usage:
usb start - start (scan) USB controller
usb reset - reset (rescan) USB controller
usb stop [f] - stop USB [f]=force stop
usb tree - show USB device tree
usb info [dev] - show available USB devices
[...]
```



U-Boot information commands

Version details: `version`

```
=> version
U-Boot 2020.04 (May 26 2020 - 16:05:43 +0200)
arm-linux-gcc (crosstool-NG 1.24.0.105_5659366) 9.2.0
GNU ld (crosstool-NG 1.24.0.105_5659366) 2.34
```

NAND flash information: `nand info`

```
=> nand info
Device 0: nand0, sector size 128 KiB
  Page size       2048 b
  OOB size          64 b
  Erase size    131072 b
  subpagesize     2048 b
  options     0x40004200
  bbt options 0x00008000
```

MMC information: `mmc info`

```
=> mmc info
Device: STM32 SD/MMC
Manufacturer ID: 3
[...]
Capacity: 14.8 GiB
Bus Width: 4-bit
```

Board information: `bdinfo`

```
=> bdinfo
boot_params = 0x00000000
DRAM bank   = 0x00000000
-> start    = 0xc0000000
-> size     = 0x20000000
flashstart  = 0x00000000
flashsize   = 0x00000000
flashoffset = 0x00000000
baudrate    = 115200 bps
relocaddr   = 0xddb21000
reloc off   = 0x1da21000
[...]
fdt_blob    = 0xdbb01950
new_fdt     = 0xdbb01950
fdt_size    = 0x0001d540
Video       = display-controller@5a001000 inactive
[...]
```

- DRAM starts at `0xc0000000`, for a size of 512 MB (`0x20000000`).
- The end of the memory is used by U-Boot itself: `relocaddr` is the location of U-Boot in RAM.



Concept of U-Boot environment

- A significant part of the U-Boot configuration happens at compile time: `menuconfig`
- U-Boot also has runtime configuration, through the concept of *environment variables*
- Environment variables are key/value pairs
  - Some specific environment variables impact the behavior of different U-Boot commands
  - Additional custom environment variables can be added, and used in *scripts*
- U-Boot environment variables are loaded and modified in RAM
- U-Boot has a default environment built into its binary
  - used when no other environment is found
  - defined in the configuration
  - the default environment is sometimes quite complex in some existing configurations
- The environment can be persistently stored in non-volatile storage



U-Boot environment persistent storage

Depending on the configuration, the U-Boot environment can be:

- At a fixed offset in NAND flash
- At a fixed offset on MMC or USB storage, before the beginning of the first partition.
- In a file on a FAT or ext4 partition
- In a UBI volume
- Not stored at all, only the built-in environment in the U-Boot binary is used

```
.config - U-Boot 2020.07 Configuration
 Environment
[ ] Environment is not stored
[ ] Environment in EEPROM
[*] Environment is in a FAT filesystem
[ ] Environment is in a EXT4 filesystem
[ ] Environment in flash memory
[ ] Environment in an MMC device
[ ] Environment in a NAND device
[ ] Environment in a non-volatile RAM
[ ] Environment is in OneNAND
[ ] Environment is in remote memory space
[ ] Environment in a UBI volume
    (mmc) Name of the block device for the environment
    (0) Device and partition for where to store the environemt in FAT
    (uboot.env) Name of the FAT file to use for the environment
    (0x4000) Environment Size
[*] Relocate gd->env_addr
[ ] Create default environment from file
[ ] Add run-time information to the environment
[ ] Block forced environment operations
```

U-Boot environment configuration menu



U-Boot environment commands

- `printenv`
  Shows all variables
- `printenv <variable-name>`
  Shows the value of a variable
- `setenv <variable-name> <variable-value>`
  Changes the value of a variable or defines a new one, only in RAM
- `editenv <variable-name>`
  Interactively edits the value of a variable, only in RAM
- After an `editenv` or `setenv`, changes in the environment are lost if they are not saved persistently
- `saveenv`
  Saves the current state of the environment to storage for persistence.
- `env` command, with many sub-commands:
  `env default`, `env info`, `env erase`, `env set`, `env save`, etc.



U-Boot environment commands example

```
=> printenv
baudrate=19200
ethaddr=00:40:95:36:35:33
netmask=255.255.255.0
ipaddr=10.0.0.11
serverip=10.0.0.1
stdin=serial
stdout=serial
stderr=serial
=> setenv serverip 10.0.0.100
=> printenv serverip
serverip=10.0.0.100
=> saveenv
```



U-Boot memory allocation

- Many commands in U-Boot loading data into memory, or using data from memory, expect a RAM address as argument
- No built-in memory allocation mechanism → up to the user to know usable memory areas to load/use data
- Use the output of `bdinfo` to know the start address and size of RAM
- Avoid the end of the RAM, which is used by the U-Boot code and dynamic memory allocations
- Not the best part of the U-Boot design, sadly



U-Boot memory manipulation commands

- Commands to inspect or modify any memory location, useful for debugging, poking into hardware registers, etc.
- Addresses manipulated in U-Boot are directly physical addresses
- Memory display `md [.b, .w, .l, .q] address [# of objects]`
- Memory write
  `mw [.b, .w, .l, .q] address value [count]`
- Memory modify (modify memory contents interactively starting from address)
  `mm [.b, .w, .l, .q] address`



U-Boot raw storage commands

U-Boot can manipulate raw storage devices:

NAND flash

- `nand info`
- `nand read <addr> <off|partition> <size>`
- `nand erase [<off> [<size>]]`
- `nand write <addr> <off|partition> <size>`
- More: `help nand`

MMC

- `mmc info`
- `mmc read <addr> <blk#> <cnt>`
- `mmc write <addr> <blk#> <cnt>`
- `mmc part` to show partition table
- `mmc dev` to show/set current MMC device
- More: `help mmc`

USB storage

- `usb info`
- `usb read <addr> <blk#> <cnt>`
- `usb write <addr> <blk#> <cnt>`
- `usb part`
- `usb dev`
- More: `help usb`

Note: `<addr>` are addresses in RAM where data is stored



U-Boot commands example

List partitions on MMC

```
STM32MP> mmc part
Partition Map for MMC device 0  --   Partition Type:
EFI

Part    Start LBA       End LBA         Name
        Attributes
        Type GUID
        Partition GUID
  1     0x00000022      0x000001d3      "fsbl1"
        attrs:  0x0000000000000000
        type:   0fc63daf-8483-4772-8e79-3d69d8477de4
        type:   linux
        guid:   72c63477-c475-4cf7-988e-b763bce4604e
  2     0x000001d4      0x00000385      "fsbl2"
        attrs:  0x0000000000000000
        type:   0fc63daf-8483-4772-8e79-3d69d8477de4
        type:   linux
        guid:   66d616db-de56-4a1e-9b13-9b1a5a6e360f
  3     0x00000386      0x00001385      "fip"
        attrs:  0x0000000000000000
        type:   0fc63daf-8483-4772-8e79-3d69d8477de4
        type:   linux
        guid:   6251ecf7-d985-4d81-a396-7a6b6fab8b7c
  [...]
```

Read block 0x22 from MMC to RAM 0xc0000000

```
STM32MP> mmc read c0000000 22 1
MMC read: dev #0, block #34, count 1 ... 1 blocks read: OK
```

Dump memory at 0xc00000000

```
STM32MP> md c0000000
c0000000: 324d5453 00000000 00000000 00000000  STM2............
c0000010: 00000000 00000000 00000000 00000000  ................
c0000020: 00000000 00000000 00000000 00000000  ................
c0000030: 00000000 00000000 00000000 00000000  ................
```



U-Boot filesystem storage commands

- U-Boot has support for many filesystems
  - The exact list of supported filesystems depends on the U-Boot configuration
- Per-filesystem commands
  - FAT: `fatinfo`, `fatls`, `fatsize`, `fatload`, `fatwrite`
  - ext2/3/4: `ext2ls`, `ext4ls`, `ext2load`, `ext4load`, `ext4size`, `ext4write`
  - Squashfs: `sqfsls`, `sqfsload`
- "New" generic commands, working for all filesystem types
  - Load a file:
    `load <interface> [<dev[:part]> [<addr> [<filename> [bytes [pos]]]]]`
  - List files: `ls <interface> [<dev[:part]> [directory]]`
  - Get the size of a file: `size <interface> <dev[:part]> <filename>`
    (result stored in `filesize` environment variable)
  - `interface`: `mmc`, `usb`
  - `dev`: device number, `0` for first device, `1` for second device
  - `part`: partition number



U-Boot filesystem command example

List files

```
STM32MP> ls mmc 0:4
<DIR>       1024 .
<DIR>       1024 ..
<DIR>      12288 lost+found
<DIR>       2048 bin
<DIR>       1024 boot
<DIR>       1024 dev
<DIR>       1024 etc
[...]

STM32MP> ls mmc 0:4 /etc
<DIR>       1024 .
<DIR>       1024 ..
             209 asound.conf
<DIR>       1024 fonts
             334 fstab
             347 group
[...]
```

Load file

```
STM32MP> load mmc 0:4 c0000000 /etc/fstab
334 bytes read in 143 ms (2 KiB/s)
```

Show file contents

```
STM32MP> md c0000000
c0000000: 663c2023 20656c69 74737973 093e6d65  #<filesystem>.
c0000010: 756f6d3c 7020746e 3c093e74 65707974  <mountpt>.\<type
c0000020: 6f3c093e 6f697470 093e736e 6d75643c  >.<options>.\<dum
c0000030: 3c093e70 73736170 642f0a3e 722f7665  p>.<pass>./dev/r
c0000040: 09746f6f 6509092f 09327478 6e2c7772  oot./..ext2.rw,n
[...]
```



U-Boot networking

- Environment variables
  - `ethaddr`: MAC address
  - `ipaddr`: IP address of the board
  - `serverip`: IP address of the server for network related commands
- Important commands
  - `ping`: ping a destination machine. Note: U-Boot is not an operating system with multitasking/interrupts, so ping from another machine to U-Boot cannot work.
  - `tftp`: load a file using the TFTP protocol
  - `dhcp`: get an IP address using DHCP



TFTP

- Network transfer from the development workstation to U-Boot on the target takes place through TFTP
  - *Trivial File Transfer Protocol*
  - Somewhat similar to FTP, but without authentication and over UDP
- A TFTP server is needed on the development workstation
  - `sudo apt install tftpd-hpa`
  - All files in `/srv/tftp` are then visible through TFTP
  - A TFTP client is available in the `tftp-hpa` package, for testing
- A TFTP client is integrated into U-Boot
  - Configure the `ipaddr`, `serverip`, and `ethaddr` environment variables
  - Use `tftp <address> <filename>` to load file contents to the specified RAM address
  - Example: `tftp 0x21000000 zImage`



Scripts in environment variables

- Environment variables can contain small scripts, to execute several commands and test the results of commands.
  - Useful to automate booting or upgrade processes
  - Several commands can be chained using the `;` operator
  - Tests can be done using `if command ; then ... ; else ... ; fi`
  - Scripts are executed using `run <variable-name>`
  - You can reference other variables using `$variable-name`
- Examples
  - ```
    setenv bootcmd tftp 0x21000000 zImage; tftp 0x22000000 dtb;
        bootz 0x21000000 - 0x22000000'
    ```
  - ```
    setenv mmc-boot if fatload mmc 0 80000000 boot.ini; then source;
        else if fatload mmc 0 80000000 zImage; then run mmc-do-boot; fi; fi'
    ```



U-Boot booting commands

- Commands to boot a Linux kernel image
  - `bootz` → boot a compressed ARM32 `zImage`
  - `booti` → boot an uncompressed ARM64 or RISC-V `Image`
  - `bootm` → boot a kernel image with legacy U-Boot headers
  - `zboot` → boot a compressed x86 `bzImage`
- `bootz [addr [initrd[:size]] [fdt]]`
  - `addr`: address of the kernel image in RAM
  - `initrd`: address of the *initrd* or *initramfs*, if any. Otherwise, must pass `-`
  - `fdt`: address of the *Device Tree* passed to the Linux kernel
- Important environment variables
  - `bootcmd`: list of commands executed automatically by U-Boot after the count down
  - `bootargs`: Linux kernel command line



U-Boot booting example

Load kernel image and Device Tree

```
STM32MP> ls mmc 0:4 /boot
<DIR>       1024 .
<DIR>       1024 ..
          117969 stm32mp157c-dk2.dtb
         7538376 zImage

STM32MP> load mmc 0:4 c2000000 /boot/zImage
7538376 bytes read in 463 ms (15.5 MiB/s)

STM32MP> load mmc 0:4 c4000000 /boot/stm32mp157c-dk2.dtb
117969 bytes read in 148 ms (778.3 KiB/s)
```

Set kernel command line and boot

```
STM32MP> setenv bootargs root=/dev/mmcblk0p4 rootwait

STM32MP> bootz c2000000 - c4000000
Kernel image @ 0xc2000000 [ 0x000000 - 0x7306c8 ]
#Flattened Device Tree blob at c4000000
   Booting using the fdt blob at 0xc4000000
   Loading Device Tree to cffe0000, end cffffcd0 ... OK
[...]
```



FIT image

- U-Boot has a concept of **FIT** image
- FIT = *Flat Image Tree*
- Container format that allows to bundle multiple images into one
  - Multiple kernel images
  - Multiple Device Trees
  - Multiple initramfs
  - Any other image: FPGA bitstream, etc.
- Typically useful for secure booting and to ensure binaries don't overlap in memory.
- Interestingly, relies on the *Device Tree Compiler*
  - `.its` file describes the contents of the image
  - Device Tree Compiler compiles it into an `.itb`
- U-Boot can load an `.itb` image and use its different elements
- https://www.thegoodpenguin.co.uk/blog/u-boot-fit-image-overview/



Generic Distro boot (1)

- Each board/platform used to have its own U-Boot environment, with custom variables/commands
- Wish to standardize the behavior of bootloaders, including U-Boot
- *Generic Distro boot* concept
- If enabled, at boot time, U-Boot:
  - Can be asked to locate a bootable partition (`part list` command), as defined by the bootable flag of the partition table
  - With the `sysboot` command, will look for a `/extlinux/extlinux.conf` or `/boot/extlinux/extlinux.conf` file describing how to boot, and will offer a prompt in the console to choose between available configurations.
  - Once a configuration is selected, will load and boot the corresponding kernel, device tree and initramfs images.
  - Example `bootcmd`:
    `part list mmc 0 -bootable bootpart; sysboot mmc 0:$bootpart any`
- https://u-boot.readthedocs.io/en/latest/develop/distro.html



Generic Distro boot (2)

Several environment variables need to be set:

- `kernel_addr_r`: address in RAM to load the kernel image
- `ramdisk_addr_r`: address in RAM to load the initramfs image (if any)
- `fdt_addr_r`: address in RAM to load the DTB (Flattened Device Tree)
- `pxefile_addr_r`: address in RAM to load the configuration file (usually `extlinux.conf`)
- `bootfile`: the path to the configuration file, for example `/boot/extlinux/extlinux.conf`

Example `/boot/extlinux/extlinux.conf`

```
label stm32mp157c-dk2-buildroot
  kernel /boot/zImage
  devicetree /boot/stm32mp157c-dk2.dtb
  append root=/dev/mmcblk0p4 rootwait
```

U-Boot boot log

```
Hit any key to stop autoboot:  0
Boot over mmc0!
switch to partitions #0, OK
mmc0 is current device Scanning mmc 0:4...
Found /boot/extlinux/extlinux.conf
Retrieving file: /boot/extlinux/extlinux.conf
131 bytes read in 143 ms (0 Bytes/s)
1:      stm32mp157c-dk2-buildroot
Retrieving file: /boot/zImage
7538376 bytes read in 462 ms (15.6 MiB/s)
append: root=/dev/mmcblk0p4 rootwait
Retrieving file: /boot/stm32mp157c-dk2.dtb
117969 bytes read in 148 ms (778.3 KiB/s)
Kernel image @ 0xc2000000 [ 0x000000 - 0x7306c8 ]
## Flattened Device Tree blob at c4000000
  Booting using the fdt blob at 0xc4000000
  Loading Device Tree to cffe0000, end cffffcd0 ... OK

Starting kernel ...
```



TF-A: Trusted Firmware



Concept of FIP

- FIP = *Firmware Image Package*
- Concept specific to TF-A
- *packaging format used by TF-A to package firmware images in a single binary*
- Typically used to bundle the BL33, i.e. the U-Boot bootloader that will be loaded by TF-A.
- https://trustedfirmware-a.readthedocs.io/en/latest/getting_started/tools-build.html
- https://wiki.st.com/stm32mpu/wiki/How_to_configure_TF-A_FIP



Configuring TF-A

- TF-A does not use *Kconfig* for configuration
- All the configuration is based on variables passed on the `make` command line
- Most variables are documented in TF-A's getting started



Configure TF-A: important variables

- `CROSS_COMPILE`, cross-compiler prefix
- `ARCH`, CPU architecture: `aarch32` or `aarch64`
- `ARM_ARCH_MAJOR`, `7` for ARMv7, `8` for ARMv8
- `PLAT`, SoC family, any directory name in `plat` that contains `platform.mk`
- `AARCH32_SP`, the Secure Payload, specific to ARMv7. Either OP-TEE or the built-in *SP-MIN* provided by TF-A
- `DTB_FILE_NAME`, path to the Device Tree describing our board
- `BL33`, path to the second stage bootloader, usually U-Boot, to include in the FIP image
- Specific to STM32MP1
  - `BL33_CFG`, path to the U-Boot Device Tree
  - `STM32MP_SDMMC=1`, enable support for SD card/eMMC in TF-A



Building TF-A for STM32MP1

```
$ make CROSS_COMPILE=arm-linux-
       ARM_ARCH_MAJOR=7
       ARCH=aarch32
       PLAT=stm32mp1
       AARCH32_SP=sp_min
       DTB_FILE_NAME=stm32mp157a-dk1.dtb
       BL33=/path/to/u-boot/u-boot-nodtb.bin
       BL33_CFG=/path/to/u-boot/u-boot.dtb
       STM32MP_SDMMC=1
       fip all
```

Build results in `build/stm32mp1/release`. Important files:

- `tf-a-stm32mp157a-dk1.stm32`, TF-A itself
- `fip.bin`, the FIP image, containing U-Boot and other elements



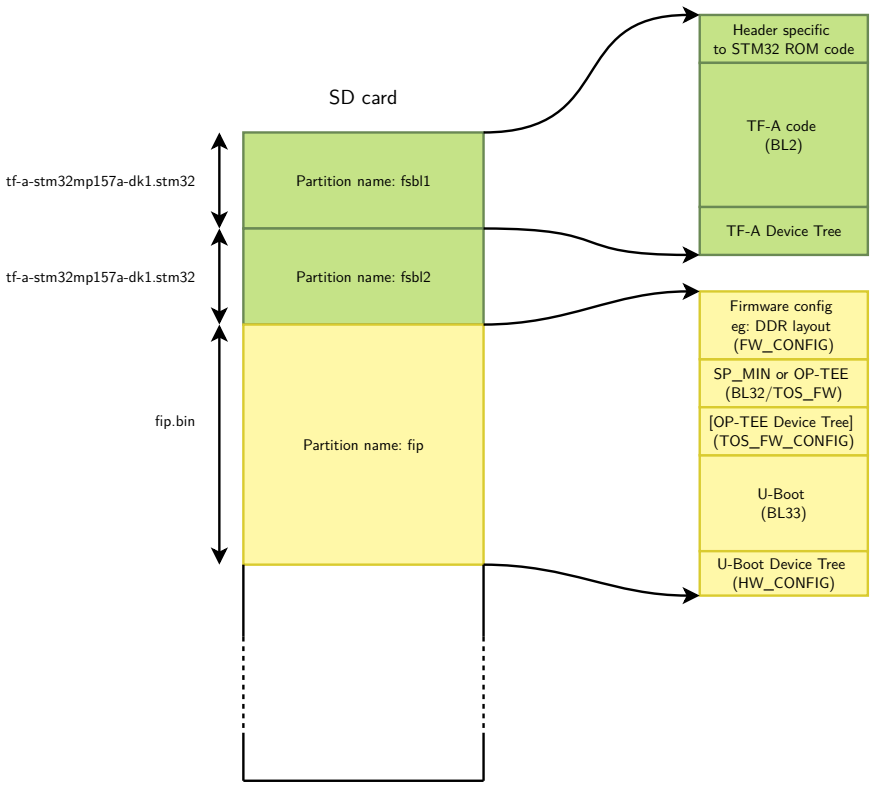
FIP image contents

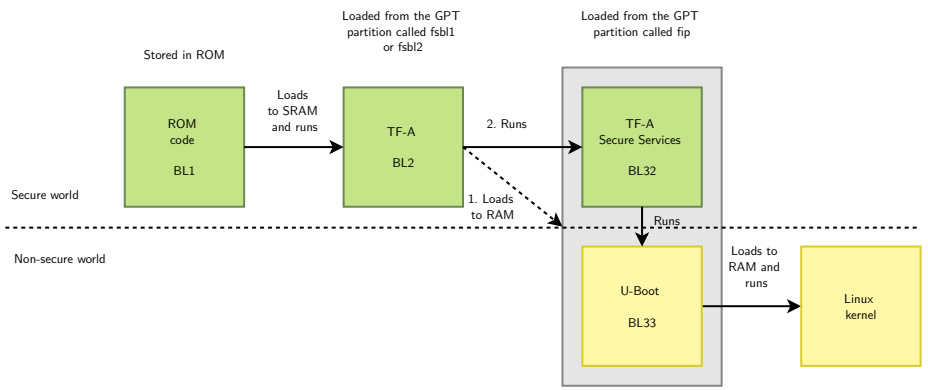
fiptool info

```
$ ./tools/fiptool/fiptool info build/stm32mp1/release/fip.bin
Secure Payload BL32 (Trusted OS): offset=0x100, size=0x8AEC, cmdline="--tos-fw"
Non-Trusted Firmware BL33: offset=0x8BEC, size=0xECE6C, cmdline="--nt-fw"
FW_CONFIG: offset=0xF5A58, size=0x226, cmdline="--fw-config"
HW_CONFIG: offset=0xF5C7E, size=0x16A98, cmdline="--hw-config"
TOS_FW_CONFIG: offset=0x10C716, size=0x3CF6, cmdline="--tos-fw-config"
```



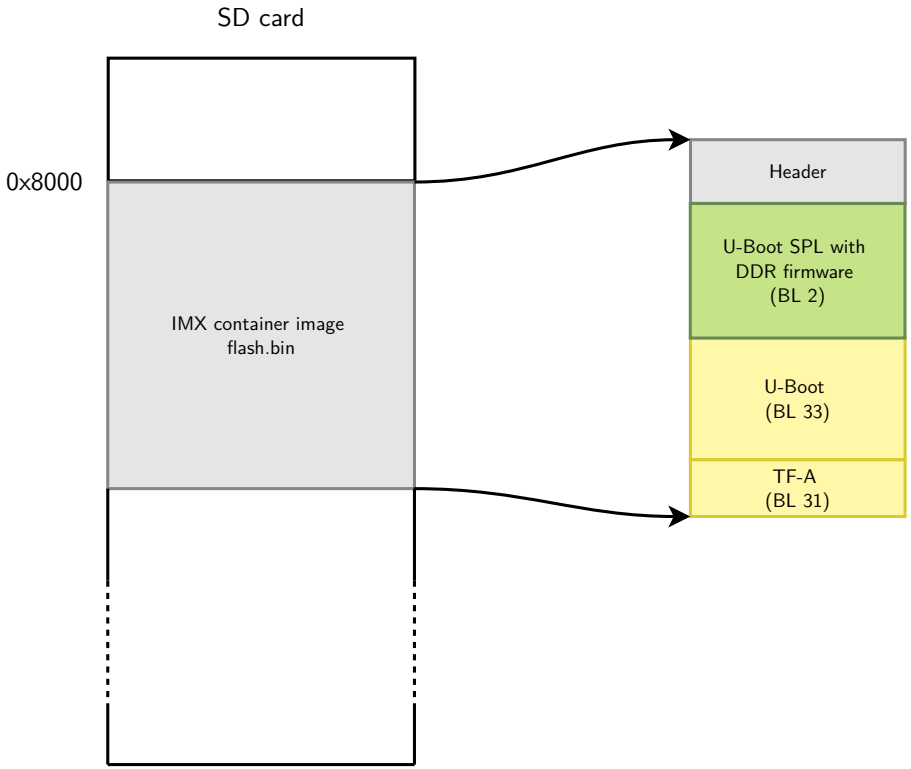
STM32MP1 partition layout

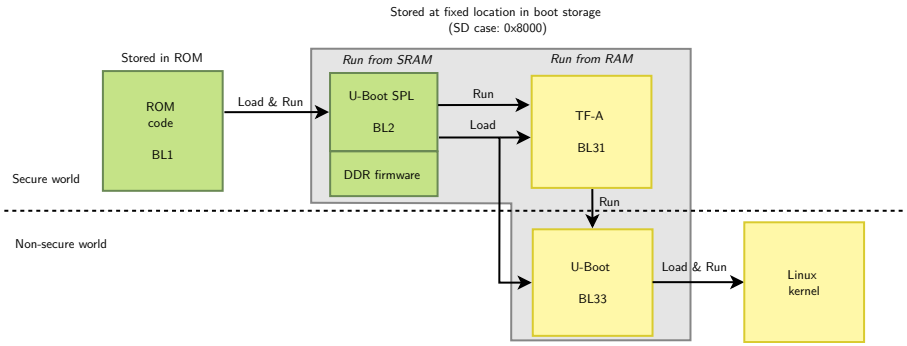




Reminder: STM32MP1's boot sequence

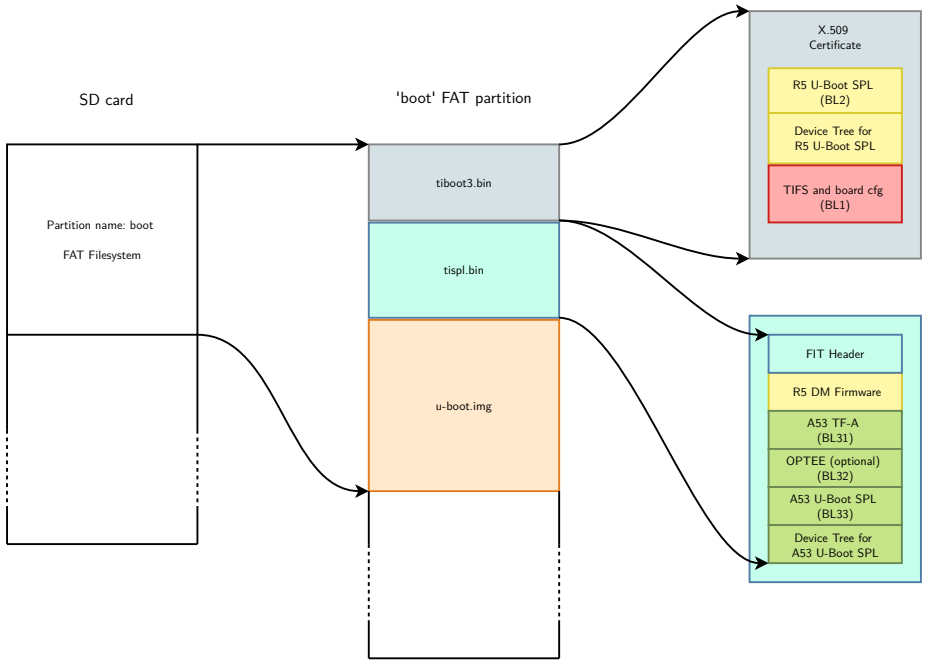
IMX93 partition layout

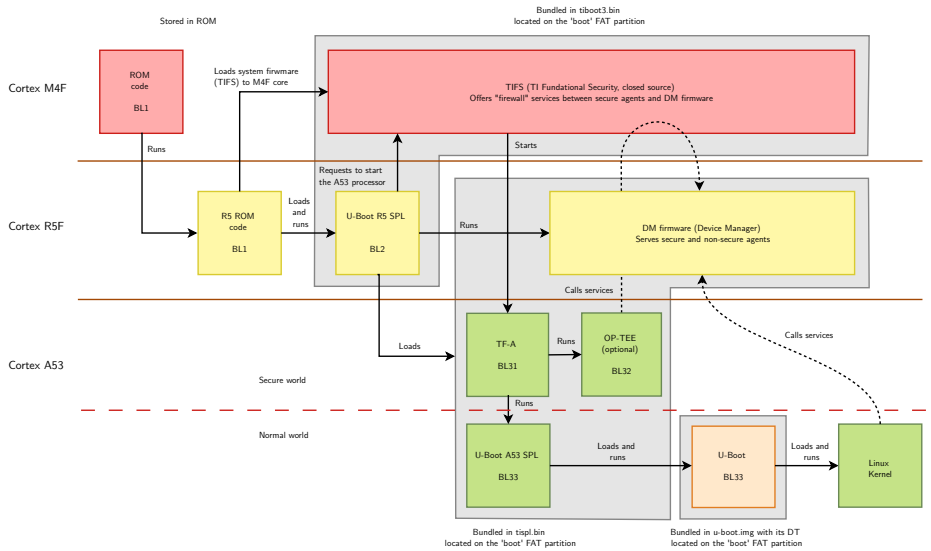




Reminder: IMX93's boot sequence

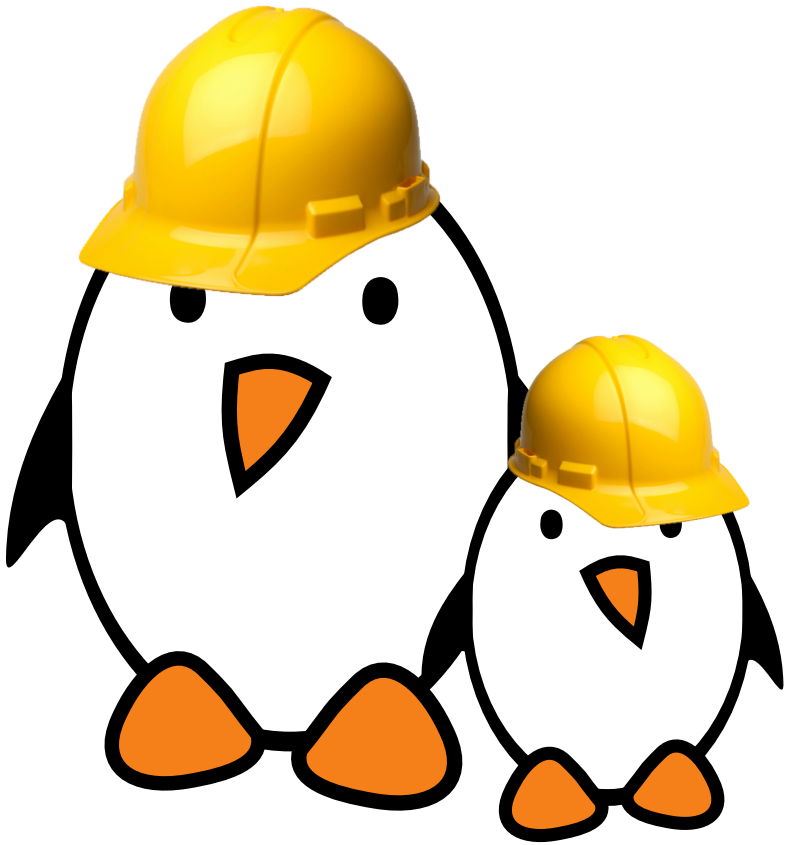
AM62x (BeaglePlay) partition layout





Reminder: AM62x's boot sequence

Practical lab - U-Boot / TF-A



Time to start the practical lab!

- Communicate with the board using a serial console
- Configure, build and install the bootloader stages:
  - *TF-A* and *U-Boot* on STM32MP1 and Beagleplay
  - *U-Boot SPL* and *U-Boot* on BeagleBoneBlack and QEMU
- Learn *U-Boot* commands
- Set up *TFTP* communication with the host